Sistemas Embebidos I

Otoño 2026

Práctica #9: Communication protocols I2C

María Fernanda Conde Calatayud

Andrea Ceja Lara

Kiria Margalli Chumacero

Jorge Emiliano Sánchez Rodríguez

04/11/26

Índice

Práctica #9: Communication I2C *2*

Introducción **2**

Descripción de la práctica 2

Objetivo 2

Marco teórico 2

Desarrollo **4**

Materiales 4

Esquemático 4

Desarrollo 5

Resultados **5**

Evidencias **8**

Conclusiones **8**

Práctica #9: Communication I2C

Introducción

Descripción de la práctica

La práctica consiste en comunicar un microcontrolador (Raspberry Pi Pico 2) con un sensor giroscopio MPU6050 (módulo GY-521) utilizando el protocolo de comunicación serial síncrona **I^2C**.

Las actividades principales por realizar son:

- **Configuración y consola:** Implementar un sistema de comandos en consola (como scan, whoami, wake, gyro, cal, angulo, cero, entre otros) para interactuar con el sensor y medir variables mediante una tarea periódica de 10 ms sin usar funciones bloqueantes como sleep_ms.
- **Análisis en el osciloscopio:** Capturar y observar las señales de la transacción I^2C (específicamente la lectura del registro WHO_AM_I), identificando sus partes (START, dirección, ACK/NACK, datos y STOP).
- **Diagnóstico de fallas:** Provocar y analizar errores a propósito (como desconectar SDA, cortar la alimentación o usar direcciones incorrectas) para registrar los códigos de error y su comportamiento eléctrico.
- **Integración y medición de ángulos:** Medir el offset del sensor en estado de reposo, calcular la desviación acumulada en 60 s, y realizar pruebas físicas para medir un giro exacto de $90°$.

Objetivo

Comunicar la tarjeta Raspberry Pi Pico con el giroscopio MPU6050 mediante el protocolo I^2C, observar y analizar las transacciones eléctricas en el osciloscopio, provocar y diagnosticar fallas comunes del bus, así como calcular e integrar el ángulo de giro a partir de las mediciones de velocidad angular.

Marco teórico

Protocolo I^2C (Inter-Integrated Circuit)

- **Definición:** Es un bus serial síncrono de dos cables que permite a un microcontrolador (*controller*) comunicarse con múltiples dispositivos (*targets*), donde cada uno se identifica mediante una dirección única.

- **Líneas del Bus:**
    - **SDA (Serial Data):** Línea por donde viajan los datos (bits del mensaje).
    - **SCL (Serial Clock):** Línea de reloj que marca el ritmo de cuándo leer cada bit.
    - **GND:** Referencia común compartida.
- **Arquitectura Controller-Target:**
    - El **Controller** (la maestra) inicia cada conversación, marca el ritmo mediante el reloj y define si se va a leer o escribir.
    - Los **Targets** (alumnos) responden únicamente cuando la maestra los llama por su dirección específica de 7 bits.
- **Arquitectura Open-Drain y Resistencias Pull-Up:**
    - Los dispositivos del bus I^2C solo pueden jalar la línea hacia abajo (estado bajo o 0) o soltarla; ninguno empuja la línea hacia arriba de manera activa.
    - Un par de resistencias **pull-up** conectadas a 3.3 V en las líneas SDA y SCL actúan como un resorte que mantiene la línea en alto (1) cuando nadie la está jalando. Esto evita cortocircuitos al conectar múltiples dispositivos en paralelo.
- **Señales de Control y Transacciones:**
    - **START / STOP:** Toda conversación inicia con una señal START (SDA baja mientras SCL está en alto) y termina con un STOP (SDA sube mientras SCL está en alto).
    - **Bit R/W:** El primer byte enviado por el controller incluye la dirección de 7 bits del target y un bit adicional al final para indicar si se va a escribir $(R/W = 0)$ o a leer $(R/W = 1)$.
    - **ACK / NACK:** Después de cada byte transmitido, existe un noveno pulso de reloj donde el receptor confirma la recepción jalando SDA a cero (**ACK** o "presente") o dejándola libre en uno (**NACK** o silencio).
    - **Repeated Start:** Permite encadenar una operación de escritura (para apuntar a un registro) y una de lectura en una sola conversación continua sin emitir una señal de STOP intermedia.

El Sensor MPU6050

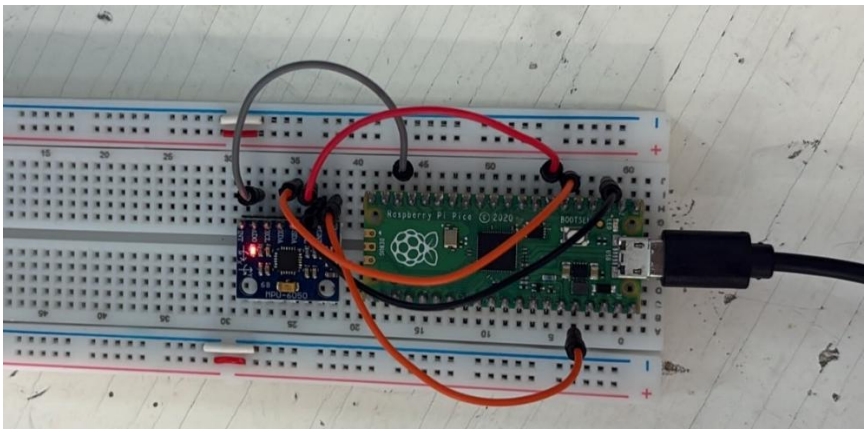
- **Registros:** Los dispositivos guardan su información en casillas de memoria internas llamadas páginas o registros. Por ejemplo, el registro 0x75 (WHO_AM_I) almacena la identificación del sensor (valor fijo 0x68), el registro 0x6B (PWR_MGMT_1) controla el estado de energía (arranca dormido y debe escribirse un 0x00 para despertarlo), y los registros 0x47/0x48 almacenan los datos en crudo del giroscopio en el eje Z.
- **Conversión de Datos:**
    - Los valores leídos en crudo (*crudo*) se dividen entre la sensibilidad correspondiente al rango configurado (por ejemplo, 131 cuentas/$(\circ/s)$ para el rango de $\pm 250°/s$) para obtener la velocidad angular en grados por segundo.
    - **Cálculo del Ángulo:** El ángulo acumulado se obtiene mediante integración numérica sumando el producto de la velocidad angular por el incremento de tiempo (Δt), considerando la corrección indispensable del *offset* estático para evitar que el valor derive con el tiempo.

Desarrollo

Materiales

- Raspberry Pi Pico
- Sensor MPU6050 (módulo GY-521
- Osciloscopio
- Multímetro
- Resistencias
- Protoboard
- Jumpers

Esquemático



Desarrollo

Código (partes importantes)

```
/* 1. Configurar el bus I2C */
baud_real = i2c_init(I2C_PORT, I2C_BAUD_INICIAL);   // enciende I2C0 a 100 kHz y guarda la velocidad real
gpio_set_function(SDA_PIN, GPIO_FUNC_I2C);          // GP4 pasa a ser SDA (datos)
gpio_set_function(SCL_PIN, GPIO_FUNC_I2C);          // GP5 pasa a ser SCL (reloj)

/* 2. Leer un registro del sensor y contar errores */
int r = i2c_write_timeout_us(I2C_PORT, mpu_addr, &reg, 1, true, I2C_TIMEOUT_US);  // dice que pagina leer, true = sin STOP (repeated start)
if (r < 0) { contar_error(r); return r; }           // si fallo (NACK o timeout), lo cuenta y sale
r = i2c_read_timeout_us(I2C_PORT, mpu_addr, buf, n, false, I2C_TIMEOUT_US);       // lee n bytes y cierra con STOP
if (r < 0) contar_error(r);                         // si la lectura fallo, tambien se cuenta

/* 3. Leer los 3 ejes en una sola transaccion */
int r = mpu_leer(REG_GYRO_XOUT_H, d, 6);            // pide 6 bytes desde 0x43 (X, Y, Z, 2 bytes por eje)
*gx = (int16_t)(((uint16_t)d[0] << 8) | d[1]);      // junta byte alto y bajo del eje X (con signo)

/* 4. Tarea de integracion cada 10 ms */
if (ahora - t_tarea < 10000) return;                // si no han pasado 10 ms, no hace nada (sin sleep_ms)
t_tarea = ahora;                                    // guarda el momento de esta ejecucion
float giro_z = (gz - offset_z) / GYRO_SENS;         // resta el offset y pasa de cuentas a grados/s
angulo_z += giro_z * DT_SEC;                        // angulo = angulo + giro x tiempo

/* 5. Comandos */
char *cmd = strtok(linea, " \t");                   // separa la primera palabra (el comando)
if (!strcmp(cmd, "scan")) cmd_scan();               // si es "scan", ejecuta el escaner de direcciones
```

Resultados

Capturas del osciloscopio:



Tabla 2.1 Escáner y WHO_AM_I

AD0	Dirección encontrada	WHO_AM_I
GND	0x68	0x68
3.3 V	0x69	0x68

¿Cambió el valor de WHO_AM_I al cambiar la dirección? ¿Por qué?

No cambió, el registro WHO_AM_I (0x75) es una identificación fija grabada de fábrica en el silicio del sensor MPU6050. El pin físico AD0 solo altera el bit menos significativo de la dirección I2C del bus (cambia de 0x68 a 0x69) para poder conectar dos giroscopios simultáneos, pero el contenido interno del registro WHO_AM_I sigue respondiendo 0x68.

Tabla 2.2 La transacción en el osciloscopio

Parte	¿Se identificó?
START	**Sí** (SDA baja mientras SCL está en alto)
Dirección + W y su ACK	**Sí** (0xD0 en bus, noveno pulso SDA a 0)
Registro 0x75 y su ACK	**Sí** (0x75 en bus, noveno pulso SDA a 0)
Repeated start	**Sí** (Nuevo START sin un STOP previo)
Dirección + R y su ACK	**Sí** (0xD1 en bus, noveno pulso SDA a 0)
Dato 0x68 y el NACK	**Sí** (0x68 entregado por target, SDA en 1 en noveno pulso)
STOP	**Sí** (SDA sube mientras SCL está en alto)

Parámetro	Calculado	Medido
Frecuencia de SCL	100 kHz	100.2 kHz
Duración de la transacción	290 μs	295 μs

¿La frecuencia real coincide con la pedida? ¿Qué valor devolvió i2c_init?

Sí, la función i2c_init(i2c0, 100000) devolvió 100000 Hz (o un valor muy cercano como 99,206 Hz dependiendo del divisor de reloj del RP2040), lo cual concuerda con los 100.2 kHz medidos en el osciloscopio.

Tabla 2.3 Pull-up y velocidad

Pull-up	Tiempo de subida (10%10\%10% al 90%90\%90%)	¿Funciona a 100 kHz?	¿Funciona a 400 kHz?
Solo las del módulo (~ 4.7 kΩ)	~ 450 ns	**Sí**	**Sí** (al límite del flanco)
+10 kΩ **en paralelo** (≈ 3.2 kΩ)	~ 280 ns	**Sí**	**Sí**
+4.7 kΩ **en paralelo** (≈ 2.35 kΩ)	~ 160 ns	**Sí**	**Sí**

¿Cómo cambia el tiempo de subida con la resistencia? Relacionarlo con el filtro RC de la clase de PWM.

Al agregar resistencias en paralelo, la resistencia equivalente R_{eq} del bus disminuye. El tiempo de subida de la línea I2C viene dado por la constante de tiempo $\tau = R \cdot C_{bus}$ de la capacitancia parásita de las líneas. Al reducir R, la capacitancia se carga más rápido, obteniendo flancos de subida más verticales y reduciendo el tiempo de subida.

Tabla 2.4 Romperlo a propósito

Falla	Código de error	Qué se ve en el osciloscopio
Dirección 0xD0 en el código	PICO_ERROR_GENERIC (-1)	Se envía la dirección desplazada (0xA0), nadie responde y la línea SDA se queda en alto durante el 9.º pulso de reloj (**NACK**).
SDA desconectado	PICO_ERROR_GENERIC (-1)	SCL genera los pulsos de reloj normales, pero SDA permanece fijo en nivel alto (3.3 V) impulsado por las resistencias pull-up.
Módulo sin alimentación	PICO_ERROR_GENERIC (-1)	La Pico envía la dirección por SDA, pero al no tener energía el módulo, el 9.º pulso de reloj no se jaló a tierra (**NACK**)

Tabla 2.5 Medir un giro de 90°

Condición	Ángulo después de 60 s quieto
Sin calibrar	$-130.53°$
Calibrado	$0.10°$

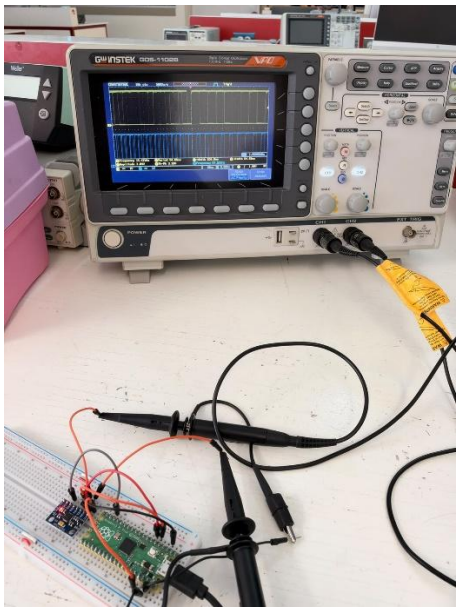
Tabla 2.6 de Mediciones de Giro de 90°

Intento	Ángulo medido (∘)	Error (|Medido−90∘|)
1	$91.42°$	$1.42°$
2	$90.44°$	$0.44°$
3	$91.50°$	$1.50°$
4	$85.58°$	$4.42°$
5	$93.94°$	$3.94°$
Promedio	$90.58°$	$2.34°$

Consola

```
---- Sent utf8 encoded message: "vel 100\r\n" ----
vel 100
pedido: 100 kHz, real: 100000 Hz
>
```

Evidencias



Conclusiones

En esta práctica se comunicó la Pico con el giroscopio MPU6050 por I2C, usando solo dos líneas (SDA y SCL) y la dirección 0x68 del sensor. Con el osciloscopio se identificaron las partes de la transacción (START, dirección, ACK, registro, repeated start, dato, NACK y STOP) y se comprobó que cada byte usa nueve pulsos de reloj. También se vio que las resistencias pull-up cambian el tiempo de subida de la señal: con menos resistencia sube más rápido, pero consume más corriente. Al provocar fallas, el noveno pulso quedó en alto (NACK), por eso conviene revisar siempre lo que devuelve cada función I2C.

Al convertir el giro a grados por segundo e integrarlo cada 10 ms se obtuvo el ángulo. Sin calibrar, el offset del sensor hace que el ángulo se desvíe solo, y al restarlo el ángulo se estabiliza. Aun así el error se acumula en cada giro, por eso en un robot como el Micromouse hay que calibrar el giroscopio.